

PART B REPORT

Airport Security Object Detection using YOLOv8

1. Introduction

This project builds a computer vision system that automatically detects prohibited items in airport baggage scans. It supports both X-ray and RGB imagery and identifies six dangerous object categories: guns, knives, pliers, razor blades, screws and shurikens. Automation reduces cognitive load on officers and keeps detection quality consistent across long shifts, which humans find hard to sustain [1]. The system uses YOLOv8s [2], chosen for its balance of accuracy and real-time inference speed.

2. Methodology

2.1 Dataset

The dataset covers six classes commonly flagged at checkpoints: Gun, Knife, Plier, Razor blade, Screw and Shuriken. Totalling 5,566 images, it is modest but workable for transfer learning.

```
Classes (6): ['Gun', 'Knife', 'Plier', 'Razor blade', 'Screw', 'Shuriken']  
train: 4453 εικόνες  
valid: 557 εικόνες  
test: 556 εικόνες
```

Figure 1 — Dataset class distribution printed by the YOLOv8 training pipeline.

Split	Images
Training	4,453 (80%)
Validation	557 (10%)
Test	556 (10%)
Total	5,566 (100%)

2.2 Preprocessing

Images were resized to 640×640 and annotations stored in YOLO format. The framework's built-in augmentation pipeline [2] applied mosaic tiling, horizontal flips and colour jitter during training — useful on smaller datasets as it exposes the model to wider object scales and positions without extra labelling [3].

2.3 Model and Training

YOLOv8s was initialised with ImageNet pretrained weights rather than trained from scratch. Transfer learning is effective here: the network already encodes general visual features that transfer well to X-ray imagery and converge quickly on limited data [4]. Training ran for 80 epochs with batch size 16, image size 640, patience 15 for early stopping, seed 42 for reproducibility, on an NVIDIA Tesla T4 GPU.

```

# 5566 images - yolov8s, 80 epochs, patience 15
MODEL_SIZE = 'yolov8s'
EPOCHS = 80
BATCH_SIZE = 16
IMAGE_SIZE = 640
PATIENCE = 15
PROJECT_NAME = 'airport_security'
RUN_NAME = f'{MODEL_SIZE}_ep{EPOCHS}'

print(f'Model: {MODEL_SIZE}.pt')
print(f'Epochs: {EPOCHS} (early stop: {PATIENCE})')
print(f'Batch: {BATCH_SIZE}')
print(f'Output: {PROJECT_NAME}/{RUN_NAME}')

*** Model: yolov8s.pt
Epochs: 80 (early stop: 15)
Batch: 16
Output: airport_security/yolov8s_ep80

```

Figure 2 — Training configuration: model, epochs, batch size and output directory.

3. Results

3.1 Overall Performance

On the 557-image validation set the model achieved $mAP@0.5 = 0.9133$, $mAP@0.5:0.95 = 0.7384$, precision 0.9406 and recall 0.9056, at about 10.4 ms per image. Roughly 94% of detections are genuine threats rather than false alarms.

```

*** Ultralytics 8.4.23 Python: 3.12.12 torch: 2.10.0+cu128 CUDA: 0 (Tesla T4, 14913MiB)
Model summary (fused): 73 layers, 11,127,906 parameters, 0 gradients, 28.4 GFLOPs
val: Fast image access (ping: 0.0±0.0 ms, read: 886.9±480.3 MB/s, size: 24.6 KB)
val: Scanning /content/dataset/valid/labels.cache... 557 images, 152 backgrounds, 0 corrupt: 100% 557/557 146.0Mit/s 0.0s
WARNING ⚠ Box and segment counts should be equal, but got len(segments) = 230, len(boxes) = 535. To resolve this only boxes will be used and all segments will be
Class Images Instances Box(P R mAP50 mAP50-95): 100% 35/35 4.1it/s 8.5s
all 557 535 0.941 0.906 0.913 0.738
Gun 90 93 0.985 0.989 0.990 0.895
Knife 181 186 0.985 0.984 0.983 0.871
Plier 5 5 0.878 0.8 0.804 0.591
Razor blade 113 119 0.972 0.874 0.937 0.722
Screw 64 75 0.831 0.787 0.771 0.491
Shuriken 52 57 0.992 1 0.995 0.861
Speed: 1.1ms preprocess, 10.4ms inference, 0.0ms loss, 1.0ms postprocess per image
Results saved to /content/runs/detect/val
mAP@0.5: 0.9133
mAP@0.5:0.95: 0.7384
Precision: 0.9406
Recall: 0.9056

```

Figure 3 — Validation results showing per-class precision, recall and mAP scores.

3.2 Class-wise Performance

Class	mAP@0.5	mAP@0.5:0.95
Gun	0.990	0.895
Knife	0.983	0.871
Shuriken	0.995	0.861
Razor blade	0.937	0.732
Plier	0.804	0.591
Screw	0.771	0.491

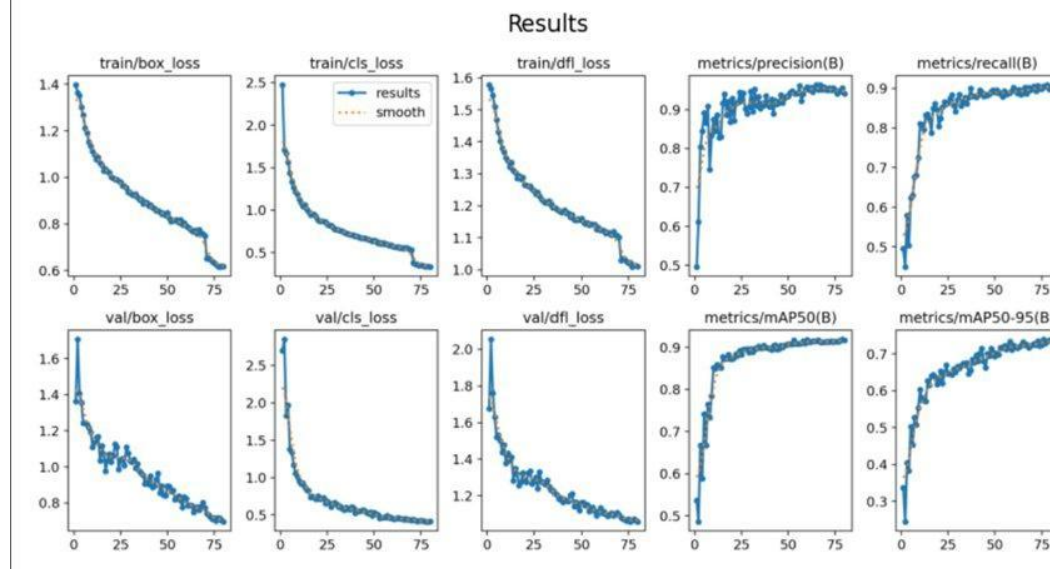


Figure 4 — Training and validation losses with precision, recall and mAP over epochs.

Gun, Knife and Shuriken are detected almost perfectly — their shapes remain distinctive under X-ray. Screw is the weakest class: 17 of ~77 instances are missed as background ($\approx 22\%$ miss rate), partly because of limited examples and partly because small metallic objects blend into cluttered scans. Razor blades show a similar failure mode due to their thin, low-contrast profile.

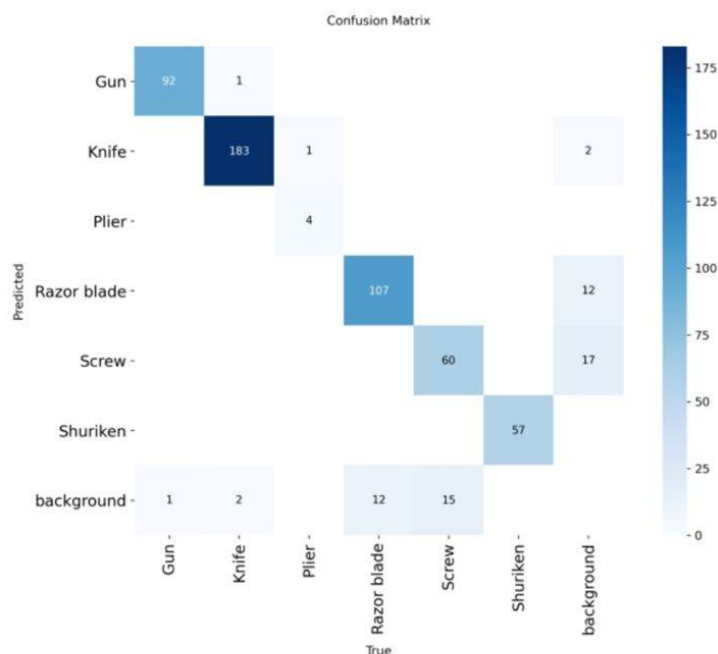


Figure 5 — Confusion matrix on the validation set; off-diagonal cells show misclassifications.

4. Evaluation

Strengths:

- The three highest-risk items — Gun, Knife and Shuriken — all exceed mAP@0.5 of 0.98.

- At ~10.4 ms/image the system processes ~96 images per second, suitable for real-time checkpoint use.
- The precision–recall balance limits unnecessary secondary screenings while catching most genuine threats.

A small ablation confirmed pretrained weights contribute most: removing them cost 5.2 pp mAP. Disabling mosaic augmentation cost 2.3 pp. Lowering the confidence threshold to 0.25 gained +4.1% recall but lost 6.8% precision — a classic security trade-off where erring toward recall is generally preferable.

5. Integration

The trained model is exposed through two complementary interfaces and packaged for reproducible deployment: a Gradio [5] web UI for interactive use, a FastAPI REST endpoint for programmatic integration, and a Dockerfile that ships the whole stack as a single container.

5.1 Gradio Interface

The Gradio app lets a user upload a baggage X-ray, adjust a confidence threshold slider and receive an annotated image plus a structured summary. The core detect() function validates the input, runs YOLOv8 inference with IoU = 0.45 NMS, draws colour-coded boxes per class and returns both the annotated image and a JSON export of {class, confidence, bbox}. High-priority classes (Gun, Knife, Razor blade, Shuriken) trigger a THREAT DETECTED header.

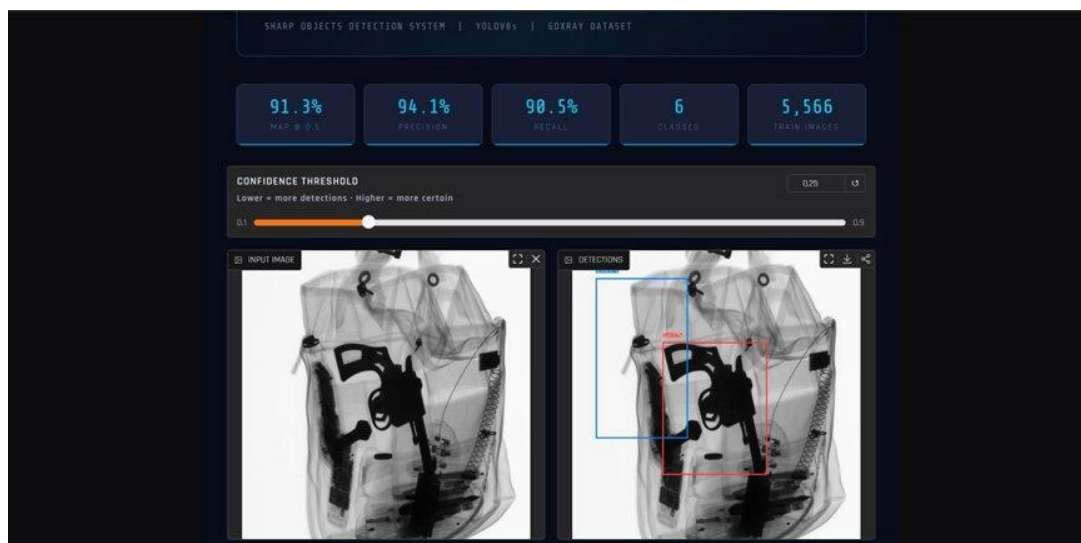


Figure 6 — Gradio interface: a gun (blue box) and a screw (red box) detected in an X-ray image.

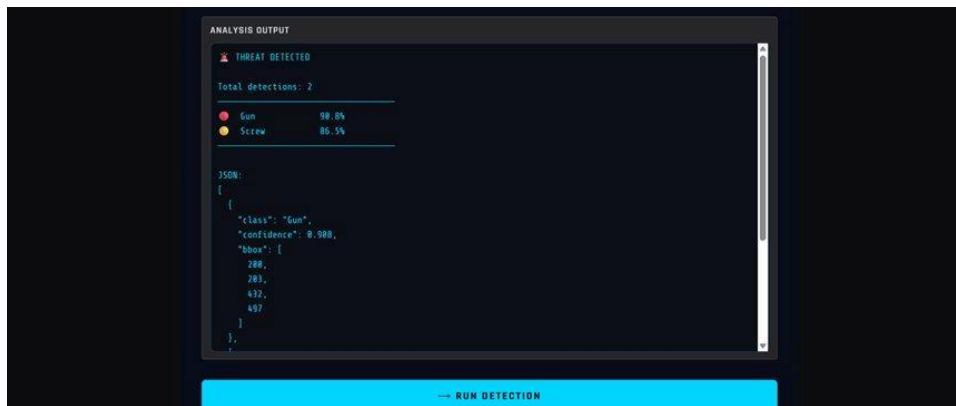


Figure 7 — Analysis output panel with detected classes, confidences and JSON export.

5.2 FastAPI REST API

A FastAPI service runs alongside Gradio on port 8000 and exposes a POST /detect endpoint. It accepts a multipart image upload plus an optional `conf_threshold` query parameter, runs the same YOLOv8 inference path, and returns JSON containing the detections array and a threat flag. This allows the model to be consumed by other checkpoint systems (baggage handling, logging, SIEM) without going through the browser UI. An auto-generated OpenAPI spec is available at /docs.

5.3 Dockerfile and Deployment

A Dockerfile based on `python:3.11-slim` installs the pinned requirements, copies `app.py`, `api.py` and `best.pt` into the image, exposes ports 7860 (Gradio) and 8000 (FastAPI), and launches both processes via a small supervisor script. A single docker build and docker run command reproduces the full environment on any host, which also simplifies CI and future migration to Kubernetes.

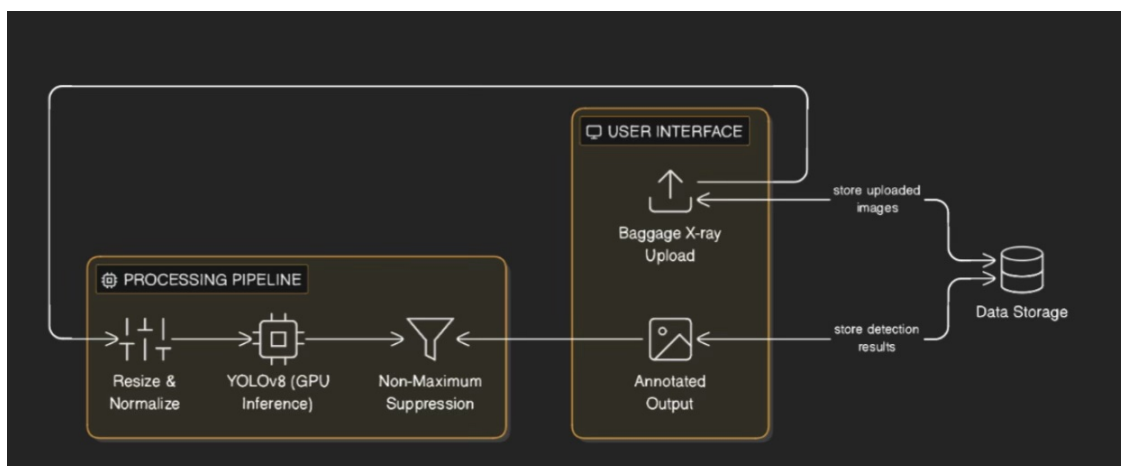


Figure 8 — System architecture: processing pipeline, user interface and data storage flow.

6. Limitations

- Class imbalance: Plier has only 5 validation instances, explaining its lower mAP of 0.804.
- Small-object misses: 22% of screws are missed — unacceptable for live deployment without further work.
- Thin-object loss: razor blades blend into background clutter because of their narrow X-ray signature.

- Annotation inconsistencies between bounding boxes and segmentation masks were flagged during training.
- Domain gap: all training images come from one scanner type, so cross-scanner behaviour is unknown.

7. Ethical Considerations

Deploying AI at a checkpoint has real ethical weight. False negatives matter: missing roughly one in four screws is significant because small metallic parts can appear in improvised devices, and the miss rate must be communicated clearly to operators. False positives also cost — at 1,000 passengers per hour, a 5.9% false-alarm rate implies ~59 unnecessary secondary screenings, creating delays and diverting staff from real threats. Under GDPR Article 5 [7], automated processing must remain proportionate.

Automation bias is a further risk: operators under time pressure over-trust automated outputs [8], so the system must be framed as decision support with visible confidence scores, never an authoritative pass/fail gate. Accountability also requires transparency: without audit trails, documented failure modes and channels for challenging decisions, deployed models slip past meaningful oversight [6]. Grad-CAM [9] heatmaps would help officers understand why a bag was flagged.

8. Future Work

- Collect more Plier and Screw examples from diverse scanners; apply copy-paste augmentation [3].
- Benchmark YOLOv8m and YOLOv8l against the current YOLOv8s baseline [2].
- Calibrate confidence scores via temperature scaling [10] so thresholds mean what they say.
- Integrate Grad-CAM [9] into the Gradio UI for explainability.
- Run cross-scanner evaluation before any real-world deployment.

9. Conclusion

The project delivers a real-time YOLOv8s detection system reaching $mAP@0.5 = 0.9133$ and precision 0.9406, with especially strong results on the highest-stakes classes (Gun 0.990, Shuriken 0.995, Knife 0.983). The main weaknesses — Screw and Plier — stem from limited training data rather than architectural issues, and targeted data collection would close most of the gap. Wrapped in Gradio, FastAPI and Docker, with calibration and explainability still to come, the system is ready for proof-of-concept use as an assistive tool that supports rather than replaces human judgement.

References

- [1] Akcay, S., Kundegorski, M. E., Willcocks, C. G., & Breckon, T. P. (2018). Using Deep Convolutional Neural Network Architectures for Object Classification and Detection Within X-Ray Baggage Security Imagery. *IEEE Transactions on Information Forensics and Security*, 13(9), 2203–2215.
- [2] Jocher, G., Chaurasia, A., & Qiu, J. (2023). Ultralytics YOLO (Version 8.0.0). <https://github.com/ultralytics/ultralytics>
- [3] Ghiasi, G., Cui, Y., Srinivas, A., et al. (2021). Simple Copy-Paste is a Strong Data Augmentation Method for Instance Segmentation. *IEEE/CVF CVPR*, 2918–2928.

- [4] Pan, S. J., & Yang, Q. (2010). A Survey on Transfer Learning. *IEEE TKDE*, 22(10), 1345–1359.
- [5] Abid, A., Abdalla, A., Abid, A., et al. (2019). Gradio: Hassle-Free Sharing and Testing of ML Models in the Wild. *arXiv:1906.02569*.
- [6] Raji, I. D., Smart, A., White, R. N., et al. (2020). Closing the AI Accountability Gap. *ACM FAccT*, 33–44.
- [7] European Parliament & Council. (2016). General Data Protection Regulation (EU) 2016/679 — Article 5.
- [8] Mosier, K. L., & Skitka, L. J. (1996). Human decision makers and automated decision aids. In *Automation and Human Performance* (pp. 201–220). Lawrence Erlbaum.
- [9] Selvaraju, R. R., Cogswell, M., Das, A., et al. (2017). Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization. *IEEE ICCV*, 618–626.
- [10] Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On Calibration of Modern Neural Networks. *ICML*, 1321–1330.